

ills and understanding of Tableau Desktop and at least three months of applying this understanding in the product. Please also view the exam guide for more details on how to prepare.

- Certified Tableau Data Analyst: This exam is for analysts that can enable stakeholders to make business decisions by understanding the business problem, identifying data to explore for analysis, and delivering actionable insights. Please also view the exam guide for more details on how to prepare.

</details>

<details markdown=1>

<summary>GitLab Training Videos</summary>

- Creating a Tableau Data Source: Custom SQL
- Creating a Tableau Data Source: Tableau Modeling
- Creating a Tableau Data Source: Virtual Connections
- Creating a Tableau Data Source: Full Walkthrough

</details>

Support

In addition to the proactive steps we've taken with self-service help resources and education initiatives, we want to provide our user community with the support they need in case these two approaches do not answer their question or resolve the issue.

<details markdown=1>

<summary>Tableau Office Hours</summary>

We hold weekly office hours (8:30-8:55 AM PST) for Tableau users to be able to regularly meet and showcase their work, as well as discuss topics / ask questions on anything relating to the tool. Please refer to our running Meeting Agenda and Question & Answer document for content covered in our sessions.

For additional troubleshooting support you can submit a support case to Tableau via the Customer Portal.

</details>

<details markdown=1>

<summary>Common Problems</summary>

Environment Problem Solution

----- ----- -----

Tableau Desktop Error FAB9A2C5 Connecting to Snowflake when using Tableau Desktop Check to see if the simba.snowflake.ini file is showing as DriverManagerEncoding=UTF-32. If it is set to 16 you'll have trouble connecting. Tableau Knowledge article.
--

| Tableau Cloud | Invalid Consent Request when opening a workbook that asks you to log into Snowflake. | If the data source for the workbook was created using OAuth, have the workbook owner republish it using the Embed password for data source feature, or alternatively ask the Data Team to switch it to using the service account instead. |

| Tableau Desktop | When publishing to Tableau Cloud from Tableau Desktop, all project folders are greyed out. | Click on the > icon next to your department's foldername to see the subfolders. You should be able to publish into those subfolders. |

| Tableau eLearning | Error Message: "Missing Authentication Cookie" | Change your browser settings as described on the Skilljar Help Center |

| Tableau Cloud or Desktop | When using "Initial SQL": "An error occurred when connecting to Snowflake" | To run your own SQL queries in Tableau, don't use the Initial SQL functionality. Snowflake doesn't support that in this context. Use Customer SQL Query instead. It will be found on the bottom left of the screen after connecting to Snowflake, picking a warehouse and schema. It's found underneath the listed tables. |

| Tableau Cloud or Desktop | Error "There was a problem connecting to the data source "Untitled Data Source"" when connecting to Virtual Connections | The password for the Snowflake service account expired. For Tableau admins: Connect to Snowflake via OAuth as the service account. It will prompt you to change the password. Afterwards, edit all the virtual connections and republish them. |

| Tableau Cloud or Desktop | Nondescript error when connecting to Virtual Connections | The underlying table changed, likely with fields removed. Ask a Tableau admin to Edit the virtual connection, find the affected table/field (check 'Alerts' when editing the virtual connection), and Exclude the deleted field. Then republish the virtual connection. |

| Tableau Desktop | Screen gets stuck on authenticating to Okta when connecting Tableau Desktop to Tableau Cloud | Use a Yubikey to connect to Okta when using Tableau Desktop. Fingerprint authentication will not work. |

</details>

<details markdown=1>

<summary>Tips and Tricks</summary>

| Environment | Tips |

| ----- | ----- |

| Tableau Desktop | Certain features are only available to Creators when using Tableau Desktop, and might not be there in Tableau Cloud. For the best experience building dashboards, use Tableau Desktop if the size of the data and the performance of your laptop allows it. |

| Tableau Cloud / Desktop | When your data set allows it, use data extracts for the best performance and end user experience |

| Tableau Cloud | Always save a newly created workbook first in your personal space before working on it. If you run into any issues while developing, the latest draft version can always be retrieved from there. If you don't it could disappear if you run into any issues. |

| Tableau Desktop | If you run into a problem where Tableau Desktop crashes, clear out the logs and try to reproduce the error. Then send the logs to @mlaanen or @tnguyen28 or post it in our external slack channel with Tableau. Follow these steps to generate the logs: Close Tableau Desktop, go to Documents > My Tableau Repository > logs, delete all the logs, Open Tableau Desktop, replicate the issue, close Tableau Desktop, open up the log files in the file path mentioned above. |

</details>

<details markdown=1>

<summary>Troubleshooting Support</summary>

1. Post your questions in the data-tableau internal slack channel to see if someone in the company has the answer.
1. Post your questions in the ext-gitlab-tableau external slack channel if you need someone from Tableau to look at it.
1. Post your questions in the Tableau Community to see if someone in the wider Tableau user community has the answer.
1. Open a support case with Tableau Support if you're experiencing a technical issue with the Tableau Cloud platform or Tableau Desktop.
 - Anyone can open a support case, however if you want expedited response and resolutions times based on Tableau Premium Support's SLAs, please reach out to the BI team in data-tableau to get support from a Tableau Admin.

</details>

Monitoring

As more and more users are onboarded and the use of analytics grows across GitLab, Tableau becomes mission-critical for data-driven decisions. Without monitoring, a "set-it-and-forget-it" deployment can be met with inadequate resources that fail to support the workload of highly-engaged users. Ongoing, proactive monitoring is required to operate and support our deployment at scale and meet the expectations of our user community.

Because Tableau is integrated with our enterprise architecture, including hardware, network, databases, and applications, understanding how everything interoperates is key for routine monitoring, performance, and troubleshooting. The monitoring function is focused on these systems and their integration with Tableau Cloud. It is primarily technical in nature and performed by IT roles. Tableau Cloud Site Administrators will work together to ensure the platform meets evolving business needs.

Tableau Cloud Status

To check the current status of Tableau Cloud and if there are any reported outages, visit the Tableau Status Page. On that page you can also sign up for notifications in the event of an outage. For reference, GitLab's Tableau Cloud instance is located in United States - West - (10AZ).

View Performance

Apdex is a standard measure of load performance. It is calculated by classifying each load event into one of three categories: Satisfied, Tolerating, Frustrated. It is based on two duration parameters: Target Time and Tolerable Time.

We use a target time of 5 seconds and follow the recommendation that the Tolerable time be four times that, 20 seconds.

The index itself is a weighted average, values range between 0 and 1, of the count of loads in each category which is then classified into five ratings: Excellent, Good, Fair, Poor, Unacceptable.

Apdex Value Range Rating
0.94 to 1.00 Excellent
0.85 to 0.93 Good
0.70 to 0.84 Fair
0.50 to 0.69 Poor
0.00 to 0.49 Unacceptable

SECTION: enterprise-data/platform/tableau/embed-demo/index.md

This is our Tableau embed demo page. It is used to demonstrate how we can embed Tableau visualizations into the handbook and provide the URL for views that are available to be embedded publicly. To facilitate uniform experience and presentation we use a shortcode to embed Tableau charts and dashboards.

To view charts embedded in the handbook the browser will need to have third party cookies enabled. This is because the handbook site needs to connect to our public Tableau site, and uses cookies to do this.

Embedding Straight Into the Handbook (No YML File)

Viz from Public Tableau site

The source url for a view from the public Tableau site can be found by looking at a list of views available for embedding.

A short video demonstrating how to do the embed.

```
{{% alert title="Attention" color="warning" %}}
```

Charts from the public Tableau site should only be embedded in the public Handbook.

```
{{% /alert %}}
```

This chart should be visible to anyone who loads the page.

```
{{< tableau toolbar="hidden" src="https://us-west-2b.online.tableau.com/t/gitlabpublic/views/OKR47EngKPITest/PastDueSecurityIssues" >}}  
{{< /tableau >}}
```

Viz from Internal Tableau site

The source url for a view from the internal Tableau site can be found by navigating to the view and selecting the Share option at the top of the screen and then selecting the Copy Link option.

This chart should only require a log in to the Tableau site and only be visible to those that have a Tableau license. GitLab team members should login via OKTA SSO.

```
{{< tableau height="600px"
src="https://10az.online.tableau.com/t/gitlab/views/OKR47EngKPITest/PastDueInfraDevIssues" >}}
{{< /tableau >}}
```

Viz with Parameters and Filters and Height

This chart should be filtered to a pre-selected Severity and Subtype. The ability to use parameters and filters means that we can more flexibly use a single chart to display different cuts in different embeds if needed.

```
{{< tableau height="600px" src="https://us-
west-2b.online.tableau.com/t/gitlabpublic/views/OKR47EngKPITest/PastDueSecurityIssues" >}}
  {{< tableau/params "Severity Select"="S2" >}}
  {{< tableau/filters "Subtype Label"="bug::vulnerability" >}}
{{< /tableau >}}
```

Embedding Performance Indicators or Other Views Using YML Files

Here is a video demonstrating the entire process of embedding PI's using yml files- public or internal views.

Passcode: 6yz?njYI

There are a few extra steps to embed any views using a yml file, because you do not add the shortcodes right into the handbook page where the actual view shows up. Here is the process.

1. First, decide if the views you are using are public, or internal. For public views in the public handbook (no sign in needed), make sure that you follow these steps first to tag the workbook as public, get it moved over to the public Tableau site, and get the URL from the list of publicly embeddable workbooks. Our license and viewer agreement requires that those public views are coming from the "public" version of our Tableau cloud. Note the process takes about 24hrs.

1. To do this, go to the workbook, select the "..." for more settings, and select "Tag". Then add "Public". Here is an example of a workbook that has the Public tag and is available for embedding.

1. You will need to make the data source an "extract" instead of "live" connection for the process to work correctly. If your extract is over 10 million rows or does not follow the guidelines for extracts, reach out to the data team to use the service account's credentials to on the workbook so that you can use a live connection.

1. It will show up in this list once it is migrated and ready. Use the share URL from here in your embedding information. Here is where to find the URL's that are available to embed from the public site.

2. If you the view is Internal (requires sign-on), you can use the normal URL of the view in

Tableau. Make sure that you get the URL by navigating to the specific view you want to embed and clicking "share" and copying that URL - the URL from the top of the page in the normal URL bar may not work.

3. Now you are ready to update your yml file. Go to the handbook page you want to update the PI's for, and edit the page. There you will find the name of the file that you are looking to update. It will look something like this:

```
md
{{</ performance-indicators "developerrelationsdepartment" //>}}
```

4. Go to the Gitlab.com repository and search "find file" for that file name, it should be a yml file.

5. Update the yml file according to these guidelines for the performance indicators YML files using a Merge Request.

Example MR's

Here is an MR that demonstrates changing the Sisense codes to Tableau codes. And here is the MR that was submitted to update the URL's in the file from internal URL's to ones from the public site.

When the first MR was submitted the assignees accidentally forgot that the views needed to be public, so they used URL's from the internal site, so the second MR was submitted to correct that error once the workbook had been tagged as Public and was available in the list of publicly available workbooks/URL's.

Workflow Recap

For private charts that require sign-in: From the handbook page, find the file name -> Search the repository for the yml file -> update the yml file to point to the Tableau workbook. Watch out for your tabs and spaces, they need to match the formatting exactly!

For public charts in the public handbook: Tag the affected workbooks as "Public" -> Get their URL from the list of publicly embeddable views -> Find the file name -> Search the repository for the yml file -> Update the YML file with an MR.

Views Available for Public Embedding

```
{{< tableau toolbar="hidden" src="https://us-west-2b.online.tableau.com/t/gitlabpublic/views/PublicSiteViews/PublicViewTable" >}}
{{< /tableau >}}
```

SECTION: enterprise-data/platform/tableau/tableau-admin-guide/_index.md

This page describes the processes used to administer the Tableau sites managed by GitLab.

Additional run books, scripts, tools and repositories related to the processes will be referenced throughout this guide.

Tableau Administrator - Role and Objectives

As Tableau Administrators, we serve as the foundation of our organization's data visualization and analytics infrastructure. Our mission is to empower teams with a secure, reliable, and user-friendly analytics environment that fosters data-driven decision-making. We bridge the gap between complex data systems and actionable business insights while upholding high standards of governance and promoting self-service analytics.

Our Five North Star Goals

1. **Performance & Reliability:** Ensure a highly available Tableau environment with optimized dashboards and stable data connections for a seamless user experience.
2. **Security & Governance:** Safeguard data integrity by maintaining security policies, ensuring appropriate access controls, and ensuring compliance with governance standards.
3. **User Enablement & Adoption:** Drive engagement through training, support, and resources that empower users to confidently analyze and interpret data.
4. **Standardization & Quality:** Establish and enforce best practices for data sources and visualizations to ensure consistency, accuracy, and trust in reporting.
5. **Business Impact:** Deliver measurable value by aligning analytics initiatives with business objectives, tracking key metrics, and demonstrating how data-driven insights enhance decision-making and efficiency.

Sites

GitLab currently maintains three different Tableau sites for different purposes:

- **Main Site:** The main site is where all of the workbook and datasource development takes place and is the site team members can request a license to use.
- **Public Site:** The public site is used to host the workbooks that are intended to be embedded in the public handbook and viewed by anyone visiting the handbook page. Team members do not have direct access to the public site and no general workbook development takes place on this site.
- **Sandbox Site:** The sandbox site is used for testing of scripts and features in an environment that will not disrupt the main site. No general workbook development takes place on the sandbox site.

Additional Sites

Tableau Cloud Manager is an administrative feature that enables the creation of up to three additional Tableau sites per account. These extra sites can be used for various purposes, including enhanced governance, staging for new content, feature testing, or secure data separation by project or team. With Tableau Cloud Manager, administrators can set up isolated and well-organized workspaces, helping to maintain data integrity and ensuring seamless management across different environments.

Tools

The TableauConMan tool is developed in house to assist in automating several aspects of administering Tableau. With orchestration in Airflow and a series of configuration files known as plans, it currently performs the following functions:

- Sync content to the public site
- Manage groups and group membership on the main site

User Management

- Main Site: For the main site team members login using Okta. An Okta - Lumos integration manages Tableau Access Requests which includes review and approvals (performed automatically via Slack), and automated Provisioning and Deprovisioning. This also includes an automated deprovisioning due to inactivity for 90+ days.
- Public Site: For the public site users login directly and do not use Okta. The number of non viewer licenses on this site is limited so any active administration on the site should be done with the Analytics Service Account.
- Sandbox Site: For the sandbox site users login directly and do not use Okta. The number of non viewer licenses on this site is limited so any active administration on the site should be done with the Analytics Service Account.

Group Management

- Main Site: For the main site groups and group membership is managed via Okta with an Access Request.
- Public Site: The groups for the public site are manually maintained. The general instructions found in the groups runbook can be followed to modify groups or group membership.
- Sandbox Site: The groups for the sandbox site are manually maintained. The general instructions found in the groups runbook can be followed to modify groups or group membership.

Tableau Cloud Sites Project Structure

- Main Site: The project structure for the main site is maintained manually and should match the folder structure found in the Tableau repository. This structure is used to enforce code reviews of changes in the production project and the MR with the requested changes should be approved before changes are made to the site.
- Public Site: The project structure for the public site is maintained manually. Many processes such as content synchronization and shareable content reports are dependent on the project structure of the site. Changes to the project structure should be made with this in mind.
- Sandbox Site: The project structure of the sandbox site does not need to be fixed to any specific structure.

Content Permissions

- Main Site: The content permissions for the main site are maintained manually and should align with the Transparency value and only limit access to content when it is strictly needed. Content permissions should only be granted to groups and not individual users.
- Public Site: The content permissions for the public site are maintained manually. Many processes such as content synchronization and shareable content reports are dependent on the project structure of the site. Changes to the project structure should be made with this in mind.
- Sandbox Site: The content permissions of the sandbox site do not need to be fixed to any specific structure.

Content Management

Main Site

Stale and Unsued Content Management

1. Purpose - Content Archival

Unsued workbooks and data sources are moved to the Archive folder to maintain a clean and efficient Tableau Cloud environment. This is an automated process which ensures that only quality, actively used content remains readily accessible, thereby minimizing clutter and enhancing the user experience. Users can quickly find and access high-quality, relevant content without sifting through outdated or unused items. Automating this process ensures consistency and accuracy, saving time and effort for the Tableau Admin team. Manual intervention is minimized, reducing the likelihood of errors.

2. Prerequisites - Automation Script

A Python script is used for this automation which follows specific criteria to identify and archive stale content. Here is a detailed explanation of the stale content identification:

1. **Workbooks** - Stale Criteria: A workbook is considered stale if it has not been created, updated, or viewed in the last 90 days.
2. **Data Sources** - Stale Criteria: A data source is considered unused if it has no connected workbooks and has not been created or updated in the last 90 days.

Exclusions and Uniqueness:

- Exclusions: Workbooks and Data sources with the Tableau tag "Do NOT Archive" are excluded from archiving.
- Renaming for Uniqueness: Before moving a workbook or data source to the "Archive" folder, its name is appended with the owner, parent project, and the current date/time to ensure uniqueness and avoid conflicts.

3. Archival Schedule

This procedure is scheduled to run quarterly, specifically at the end of each quarter. The Tableau Admin team will manually kick off the process, ensuring that the automation runs at the most opportune time and any immediate concerns can be addressed.

4. Exclusion Tag

To prevent critical content from being archived, the business can use the exclusion tag "Do NOT Archive". Any workbooks or data sources with this tag will be skipped during the archiving process. The Data team may periodically review the dashboards with this tag to ensure the exceptions are still warranted. It should be noted that workbooks tagged with "Public" should also be excluded.

5. Archive Folder Access

The "Archive" folder has restricted access, meaning that the business cannot view or access its contents directly. This restriction helps in removing archived content from search results, thereby maintaining a clean and efficient environment. If users need access to any archived content, they can reach out to the BI Team via the data-tableau Slack channel for assistance.

Public Site

Stale and Unsued Content Management

Content archival and management is performed manually on an as need basis.

Sandbox Site

Content archival and management is performed manually on an as need basis.

Connected Applications

Only the public site has a connected application that is used for viewing content embedded in the public handbook. There are keys, secrets, and ids from this connected app that must be updated and shared with the handbook team: specifically the cloud function that authenticates the viewer on page load.

Tableau Cloud and Tableau Desktop Release Management

Our Data and Analytics team manages new releases for Tableau Cloud and Tableau Desktop to ensure that these updates meet company standards for stability and compatibility with our existing workflows and integrations. This process outlines our review, inspection, validation, and communication procedures for each release, along with instructions for updating Tableau Desktop.

Release Review and Validation Process

- New Release Notification: The Tableau Admin team will monitor for new Tableau Cloud and Tableau Desktop releases as they are announced by Tableau.

- Release Review and Inspection: After release the team will take two weeks to review release notes, inspect new features, enhancements, and security updates to assess potential impacts on our Tableau Cloud environment, data connections, and Tableau Desktop compatibility. Release notes can be followed in the GitLab Tableau Quarterly Update Review Epic.

- Validation and Testing: New releases will undergo a series of validation tests within a staging environment. This includes checking compatibility with our embedded data sources, dashboards, integrations, and any custom configurations.

- Communication to the Business: After a release is validated, a communication will be sent via the Slack Channel: data-tableau. Items to be covered in the release will include:

- Key features and enhancements in the new release
- Any changes that may impact user workflows
- Steps for updating Tableau Desktop

SECTION: enterprise-data/platform/tableau/tableau-developer-guide/_index.md

Quick Links

- Tips and Tricks
- Tableau Style Guide

Data Source Types in Tableau

There are a few different options for what kind of data source you can use in Tableau, and which option you pick could impact the performance & end-user experience of your dashboard. Let's get some terms defined:

- Extract vs. Live: An extracted data source has an extract of the data set that lives in Tableau. A live data source will query the underlying data source (Snowflake, Google Sheets) every time you change a filter or navigate to a new dashboard tab. Extracts will almost always perform faster.

- Local, Embedded, and Published: In the context of this tip, a ·local· and ·embedded· data source are the same thing. These are connections where the data source lives ·inside of· or ·with· the workbook. The only way to view or edit this data source is to open the workbook. This connection type can only connect to / live inside of one workbook at a time.

- Published: A published data source is published separately from the workbook. So in Tableau Cloud, you could navigate to a link for the data source, and separately a link for the workbook. You can connect one published data source to as many workbooks as you want, as it exists independently from the workbook.

A workbook's data source can be Local + Live, Local + Extracted, Published + Live, or Published + Extracted.

Tips Regarding Connection Types

You may find that developing a workbook in Tableau Desktop that uses a Published Data Source can be a slow and clunky experience. For whatever reason, a published data source connection can work slowly. To get around this problem while still using a published data source, you can work on a local copy.

This video described the process. It is an old video, so the user interface is a little outdated, but the process of creating a local copy is the same.

The written instructions are as follows: Open the workbook in Desktop and navigate to any worksheet. Then right click on the data source and select "create local copy". Next, right click on the original published data source, and select "Replace data source" from the dropdown. In the pop-up, replace your original data source (published) with the new one (local copy).

Then you can develop as you normally would. When you are finished, you can publish the data source back to Tableau Cloud, and it will turn the local copy back into a published data source connection. Finally, publish the workbook.

Be mindful that you do not publish over someone else's existing data source, with changes that may disrupt the workings of their workbook.

Editing Fields In A Published Data Source

If you are developing in a workbook (in Tableau Cloud or Desktop) which is connected to a Published Data Source, you cannot make any changes to the published data source. This means:

1. If you want to make any edits to a calculated field, you will not be able to. You will have to create a copy of the calculated field.
1. If you use the "replace references" function it will only replace the instances where the field is on a worksheet independently. It will not replace any instances of that field that are contained within another calculated field (see point number 1, you cannot edit a calculated field in a published data source)
1. If you change any of the parameters in the workbook, Tableau will create a duplicate copy of your parameter and replace your parameter with the copy in the published version. If a stakeholder comes to you and says "this dashboard is broken! The dropdown does not work!", it is probable that there are two versions of the 'broken' parameter.
1. You cannot add a Table Calculation onto a pre-existing calculated field. You have to make a copy of the calculated field, and then you can use a table calculation with the copy.

If you find yourself in the scenario where you do need to do any of the above, it is possible. There are two options - you can either follow the instructions in the section above to create a local copy of the data source, make your adjustments, and then republish the data source.

Or, you can navigate to the published data source in Tableau Cloud, open it in "edit" mode, make your changes, and republish the data source.

Data Source Approach

In general, our recommended approach to creating data sources in Tableau is to create all joins in dbt/Snowflake to materialize a final mart and/or rpt table that can be directly consumed by

Tableau for the dashboard with no further joins, relationships, or calculations required in the BI layer.

During the Tableau implementation, we experimented with the approach of bringing fact & dimension tables into Tableau and creating the joins and relationships there. However, we prefer to create the data structures in dbt/Snowflake where possible for the following reasons:

- By keeping business logic in the enterprise data warehouse as a rule and simply publishing the final mart/rpt tables for consumption, we avoid applying contradictory business logic in custom SQL across Tableau data sources.
- This approach keeps all joins & reporting logic in the data-team/ version control via the dbt/git integration.
- This approach makes the final data source query-able using SQL; this makes it easier for an Analyst to compare the results in the dashboard and the final dataset against upstream tables, and makes it more accessible for an Analyst to troubleshoot logic when a dashboard is showing unexpected results.
- This enables the same dataset to easily be used in a Tableau dashboard as well as an ad-hoc query, one-off export/analysis, or pulled into another downstream tool if needed (i.e. a Jupyter notebook).

For the same reasons, our approach to creating calculated fields is to create them in dbt/Snowflake rather than in Tableau when possible. One notable exception is for ratio metrics (for example, gross profit % would be calculated in Tableau so that it could be dynamically weighted as filters are applied to that metric; however, the numerator & denominator should both be defined in dbt/Snowflake with only the simple division occurring in Tableau).

This approach is intended to fulfill the majority of data source use cases, and Tableau developers are encouraged to try this approach first rather than creating joins & relationships in Tableau. If you find an instance that this approach does not support, please submit an MR to update this handbook guidance to share the scenarios where an alternate approach would be better.

Connection Types in Workbooks

When you are publishing a workbook or data source, there are a few Authentication options. The default options will allow you to publish your work, but you will soon be getting:

1. Messages from users that the OAuth token has expired and they are unable to access a workbook
1. If you schedule an extract, you will get emails that the extract has failed.

The way to make sure users can always access the data within a workbook, is to choose the non-default authentication option.

<details markdown=1>

<summary> In Desktop </summary>

Publishing a Data Source From Desktop

If you are publishing a data source from Desktop to the Cloud/Online to be a Tableau Published Data Source, you will get the following window:

Select the "Edit" button it/data-team/ "Authentication". It will bring you to the following pop-up. Choose your <rolename> to embed.

Publishing a Workbook With a Local Connection

A workbook with a local connection is one whose data source is living inside of the workbook, and is not a separately published Tableau Data Source which you can search Tableau Online for. When you try to publish a workbook with a local connection, you will see the following window:

Select "Edit" it/data-team/ "Data Sources" and find "Authentication". Choose to embed your <rolename>.

</details>

<details markdown=1>

<summary> In Cloud/Online </summary>

Publishing in Cloud/Online

If you are editing a data source in Cloud/ Tableau Online/ your web browser (all the same thing), in order to make sure your credentials are embedded, find "Publish As":

In the following window, be sure to check the box for 'Embed Credentials'.

</details>

Workflow for Embedding Your Rolename to Avoid Errors in Published Dashboards

This is the process for ensuring that your rolename is properly embedded into your published dashboards. This has two key steps, and following them in this order can help avoid the following two errors:

1. A user tries to access your published dashboard and is instead met with an error window asking them to login to Snowflake.

1. Another analyst tries to quickly check your data source in Cloud, to see how it is built or to see how the custom SQL is constructed.

The first place that you get the option to embed your rolename is when you first form a connection to your datasource. It looks like this:

If you want other people to be able to access your data source, you need to leave it blank. There is no reason to enter your rolename at this step, you will do it at a later step, so the proper workflow is to leave it blank at this step.

From here, set up your datasource and develop as you would like. Then, when you are ready, publish your workbook/datasource. This is where you will follow the steps from the start of this section for embedding your rolename as you publish the workbook.

If you forget to embed your rolename at this step, then your users will be asked to sign into Snowflake or otherwise send an error instead of letting them access the dashboard.

Creating Connection Types that Allow Others Without Snowflake Access to Edit the Workbook

Using a published connection, or a local extracted connection in your workbook will give the ability for others who do not have Snowflake access to edit your workbook. If a workbook is published using a local, live connection then any Explorer who wants to make edits to your workbook will need to sign-in to Snowflake, using their own credentials. This is the screen that an editor would be met with when trying to edit a workbook with a local, live connection.

In the niche use-case that you want an Explorer without Snowflake access to be able to make small edits to your workbook, ensure you use only pulished, live connections, or extract the data.

Common Connection Errors and How to Solve Them

Are you or one of your users experiencing an error screen when trying to access Tableau? See if the issue is on this list, and how to resolve it.

1. Issue: Authentication

Error:

> "Confirm that you have provided valid credentials for this data source. Tableau detected that your OAuth refresh token is expired. Reauthenticate with new credentials. Ask your Tableau admin if you need help" and "This sheet uses data that's on a Snowflake database. You need to sign into that server".

This error message can be challenging to diagnose, as it can be the result of numerous causes. Unfortunately, there is no easy way to diagnose which possible cause is the reason you are seeing this message when it occurs.

Cause	Solution/Prevention
----- -----	
The developer used a local, live connection	Follow these steps.
The last person to publish changes to the workbook/data source forgot to embed their credentials when publishing.	Follow these steps when publishing.
The credential is truly expired (it was published weeks ago)	Have the data source owner refresh their credentials following these steps.

2. Issue: Insufficient Data Access

Error:

> Upon trying to sign in to view a dashboard, you receive the error message "invalid consent request".

This generally occurs because you do not have access to something you are trying to view. This can be:

- The a database or table the view is built off of.
- The credentials the developer embedded into the workbook. See [here](#).

Solution: If the last person to publish the data source / workbook followed the steps linked above for correctly embedding their credentials, then it is likely an access issue. The simplest solution would be for the developer to publish the connection as a published and extracted data source.

If you need access and using an extracted connection is not suitable, you can check with the developer if they used a "large" warehouse which you do not have access to, or if there are non-standard tables used in the workbook that have restricted access.

3. Issue: Missing Columns in Data

Error:

> "An unexpected error occurred. If you continue to receive this error please contact your Tableau Server Administrator" combined with "TableauException: ERROR: The field '[name]' in the datasource 'sqlprox.' does not exist in your database. It was either modified or removed. Would you like to reset the view?".

This indicates that the connection is looking for a column that does not exist/ is broken. It may have been dropped or modified in Snowflake, and now those changes are causing a breakage in Tableau.

Solution: Contact the owner of the workbook for assistance. The easiest way to resolve errors like this is by downloading a local copy of the data source or workbook to Tableau Desktop and deleting or replacing the fields there.

Embedding in the Handbook

In order for views from workbooks to be embedded and viewable in the handbook, public or

internal, the workbook and their data sources must be prepared in a specific way. To be embedded in the public handbook the workbook and relevant datasource must be copied from the internal GitLab Tableau site to the public GitLab Tableau site. To facilitate correct viewing of embedded views and the synchronization of content to the public site, workbooks must be set up in a specific way and given a specific tag. Views that are meant to be embedded on the internal site do not need to be in a specific project, but should still meet the set up guidelines.

Instructions for how to embed a Tableau chart can be found on the Handbook Embedding Demonstration page.

Workbook Set Up

For embedding in the handbook, views will embed better than dashboards will, so each view that is meant to be embedded should be designed to function without user selected inputs. Filters and parameters can be preset during the embedding process, but will not be changeable by the viewer. Additionally, the view must meet the following criteria:

- The view cannot be hidden
- For embedding in the public handbook each datasource must connect to Snowflake with a Data Team Service Account username and password or use an extract
- For embedding in the public handbook each workbook must have the Public tag.

Data Source

If you are not using an extract, like when your extract is going to be over 10 million rows, then you will need to use the Data Team Service Account's credentials. Reach out to the data team to get set up with those credentials.

Be mindful when you are embedding your credentials in the data source while publishing either internal or external views. Using an extract with your role embedded will be the clearest way to make sure that users can always view the data and will not experience an authorization expiration error.

Make sure that if you do use the Data Team's credentials to publish the workbook, when you make any changes to the workbook it retains those credentials. You will need to embed passwords in the data source for the views to show correctly. This box may come unchecked when you are making changes.

Public Tag

If your view is public and embedded in the public handbook (aka, people do not need to sign-in to view it), then it needs to be on the Public GitLab Tableau Cloud site due to the viewer license agreements. To tag a workbook as public, click on the workbook. On the main page for the workbook where you can see each of the views, next to the name, there is a "more settings" option '...'. Select that, and find "Tag...". Here, you can add "Public" as a tag.

It will take about a day for the URL to show up in this list. Once it does, copy that URL and use it in the embedding information. If your view has not shown up after a day or so, it is likely because one of your data sources is not following the guidelines of A\ being an

extracted connection or B\ using the data team's service account's credentials.

Workbook Synchronization

Each workbook with views that are meant to be embedded in the public handbook must be tagged with the Public tag. This will ensure that the workbook, and their datasources are copied to the public GitLab Tableau site. Only Creators and Explorers who can access the workbook can tag the workbook, see the Tableau documentation for more information. The individual tagging must it/data-team/stand if the data should be shared publicly and if there is any question please work with the BI team to check and apply the tag. Removing this tag from a workbook will delete the workbook from the public GitLab Tableau site, this will cause handbook pages trying to load a view from that workbook to display an error. It should be noted that it can currently take up to 48 hours for the synchronized workbook to show up in the list of views available for embedding.

Workbook Naming Convention

When publishing workbooks to our Tableau Cloud site for the first time please name the workbook with their intended / official title, so that the resulting URL will capture just this title (this will allow us to keep the same URL when the workbook is published to the Production spaces):

Publishing to the Development project:

All workbooks published to the Development project will be attached the Draft and their department tags to indicate that they are in development mode and not a workbook that has been peer reviewed and intended to serve as the single source of truth (SSOT) for a use case. The BI team will leverage the Tags functionality available in Tableau Cloud to better organize workbooks by department and publishing status. For example, this workbook below is assigned the Draft and Data Team tags:

To add tags to the workbook select on the ellipse symbol to the right of that workbook and click on Tag...:

Once in the Tag window, add in the Draft and department tags for the workbook:

Publishing to Tableau Cloud

There are two environments for publishing: Development and Production.

- Development is intended for testing and iterating on dashboards and data sources. This environment allows for experimentation and refinement before content is finalized.
- Production is the environment for deploying finalized, validated content that is ready for broader distribution and trusted use across the organization. Only content that has passed all necessary reviews and meets quality standards should be promoted to Production. To publish content to Production, follow the workflow diagram below. You can request the promotion of a workbook or data source to Production by creating a Tableau issue and using the template: Tableau Publish to Production. This process ensures all necessary steps are taken to safeguard data integrity and accuracy.

Procedure for Publication to Production

mermaid

flowchart LR

A["1.Developer publishes Tableau workbook or data source to a Development Folder"]

B["2.Developer creates a

C["3.Developer confirms the content meets promotion criteria"]

D["4.Department Project Lead reviews issue and validates"]

E["5.Project Lead Publishes to Production"]

A

subgraph Create Content

A

end

A ---> B

subgraph Verifications and Approvals

B ---> C ---> D

end

D ---> E

subgraph Deploy

E

end

Tags

Applying tags allows us to provide more information on the workbook, so that we can easily discern them by their business function / department and distinguish draft content that is still in development. To filter workbooks by their tags, please click on the search box in the upper right-hand corner of the project. [it/data-team/](#) Content types select Workbooks:

Once in the Workbooks section, click on the Tags dropdown to filter content by tags:

Workbook and Data Source Descriptions

Adding a description to a workbook or data source in Tableau enhances clarity and usability by providing context and helping users quickly grasp the content's purpose. Descriptions should be concise, ideally 1-2 sentences, and clearly outline the intended use case for the content.

How to add a description:

1. Navigate to the Workbook or Data Source

1. Click on the three dots (...) next to the workbook or data source name and select Edit Details.

1. In the Edit Details page, find the Description field. Enter the description you want to associate with the workbook or data source.

1. Once you've added the description, click Save to apply the changes.

Performance Indicators

The full code for embedding performance indicators into the handbook is typically not found on the same page as where the actual indicators are displayed. Instead, you might find something like this:

Performance Indicator Shortcode

```
{{/% performance-indicators "developerrelationsdepartment" /%}}
```

In order to update the performance indicator, you need to find the yml file which is associated with the performance indicators seen, and you can update from there. In order to locate the yml file, look at the name of the file that you see in the shortcode. In the example above, you would be looking for developerrelationsdepartment, which is the title contained within " ".

To find this file, you are going to go to the GitLab-com repository, which is a repo "for the public-facing marketing website of GitLab, including improvements to the docs and the handbook". From the repository, find "Find File" and then paste in the name of the file you are looking for. In this example, you would paste developerrelationsdepartment.

This will bring you to the yml file which you are looking for. From here, you can follow the instructions below to modify the file to include the Tableau view (dashboard or sheet) which you are looking for. Be sure to follow the Embedding Instructions when embedding views.

Two reminders, first - make sure that any public views (does not need login access) that are embedded into the public handbook are coming from the public Tableau site. This means that the workbook they come from has been tagged "Public", and you are getting the URL from the views available for embedding page. More information on this process can be found on the Handbook Embedding Demonstration Page.

Second, if you are embedding a non-public view (requires login), make sure to copy the URL from the "share" button on the top right of the view, not the URL at the top of the page.

YML

The data/performanceindicators.yml file in the handbook repositories is the basis for a system that automatically generates handbook pages with performance indicator content on them. The structure can take a list of charts and each chart can take a list of filters and parameters. Only charts not tagged as public should be included on internal handbook pages. The following is an example of how to add the needed information to the data file:

yml

```
- name: MR Rate
  description: MR Rate is a monthly evaluation of how MRs on average an Development engineer performs.
  tableaudata:
    charts:
      - url: https://us-west-2b.online.tableau.com/t/gitlabpublic/views/OKR47EngKPITest/PastDueSecurityIssues
        height: 300px
        toolbar: hidden
        hidetabs: true
        filters:
          - field: Subtype Label
            value: bug::vulnerability
        parameters:
          - name: Severity Select
            value: S2
    iskey: true
```

For additional clarity, the exact syntax of this code block is very important so that it can be read like JSON data. Sometimes you might be updating an existing handbook yml file with everything filled out, except the chart is currently a Sisense chart. To replace the Sisense chart, replace the lines of code which point to the Sisense chart. You can leave everything else in the file as-is.

If you just want to embed a chart without any height, filter, or parameter specifications you would use the following:

yml

```
tableaudata:
  charts:
    - url:
```

Which, in plain English, is:

yml

```
(tab)tableaudata:
(tab)(tab)charts:
(tab)(tab)(tab)-(space)url:
```

Please note, a tab is two spaces.

Row Level Security

To use Row Level Security within Tableau the developer will need to use an entitlement table, found in prod.entitlement. The entitlement table will be joined with the appropriate source table using the Tableau Data Modeling interface; this ensures that the tables can be properly filtered at query time and will allow for extracts to properly implement row level security. Once the entitlement table is joined to its corresponding source table, a data source filter needs to be added to ensure the rows are correctly filtered to the current user.

Example

Find the correct entitlement table for the table you are using. The entitlement table should be named similar to the table you wish to join it to.

Perform a direct inner join, not a relationship, between the source table and the entitlement table.

Create a Data Source filter using the USERNAME() function and the tableauuser field in the entitlement table. This is the step that will ensure that only rows visible to the current user will be retrieved.

Geo-Based Row-Level Security

Implementing RLS based on GEO data in Tableau ensures that users access only the data pertinent to their assigned GEO. This is facilitated through the entsfdcgeopubsecsegment table which integrates SFDC user roles with Tableau.

This Geo based entitlement table is designed to manage user access to specific GEOs by combining information from SFDC user roles and Tableau's SAFE access groups. The table's logic determines access based on the following criteria:

- Users with Assigned GEOs: If a Salesforce user has an assigned GEO, they are granted access to that specific region.
- SAFE and SFDC Role Users: Users who are part of both the Tableau SAFE access group and specific Salesforce roles gain global access. (e.g. Executive)
- Non-Pubsec Roles: Certain roles are granted global access excluding the 'AMER-PUBSEC' GEO

(e.g. Executive - Global Minus Pubsec)

- Non-SFDC SAFE Users: Tableau SAFE users without corresponding Salesforce roles are granted access based on their Tableau entitlements.

This approach ensures that each user's access to GEO data aligns with their organizational role and permissions.

Submitting an Access Request (AR) to Update Roles

If you require modifications to your Salesforce roles or Tableau access groups to change your GEO access permissions, follow GitLab's standard Access Request process:

- Submit the AR: Complete the standard AR issue template with the necessary details, specifying the roles or access levels you wish to obtain or modify.
- Approval Workflow: Your request will undergo the standard approval process, involving reviews by the relevant managers and system administrators.
- Role Assignment: Upon approval, the Sales Systems team or the Tableau Admin will implement the changes in Salesforce and/or Tableau.
- Entitlement Update: The entsfdcgeo table will automatically reflect these changes, updating your data access permissions accordingly.

Guidelines for Publishing Extracts

1. Data Source Live vs. Extract - Extracts are primarily a performance optimization tool and should not be the default choice. Use Live Connections by default, and consider Extracts only in the following situations:

- Dashboard Performance - If visualizations take longer than one minute to load despite applying performance optimizers.
- Heavy traffic data sources - To reduce data warehouse costs, use scheduled Extract refreshes instead of numerous user live queries.

1. Extract Size - We have 1 TB of storage on Tableau Cloud. Extracts should not exceed 5 GB, which accommodates up to ~50 million rows of data.

1. Extract Scheduling - Schedule Extract refreshes between 18:00 and 05:00 UTC. Ideally, limit refreshes to business weekdays.

1. Optimize Extracts - Consider the following strategies to shrink and improve extract refresh performance:

- Aggregate Data - Aggregate data at a higher level before creating an extract to reduce size and improve performance.
- Filter Data - Apply filters to include only relevant data in the extract. This can help

reduce size and improve refresh times.

- Use Incremental Refreshes - For large datasets, configure incremental refreshes instead of full refreshes. This updates only the new or changed data, which is more efficient. Consider using Tableau Prep to implement incremental and merge/update refreshes.

1. Extract Refresh Suspension - Tableau Cloud will automatically suspend data sources that remain unused for 30 days.

Improving Local Connection Timeouts

To reduce the number of times Tableau Desktop will ask to reconnect, the developer can set their local snowflake driver to keep their session alive.

To do this the developer needs to edit the odbc.ini file and set the CLIENTSESSIONKEEPALIVE flag to True. Typical locations for the file can be found in the Snowflake documentation

Replacing Datasources in Tableau Desktop

The steps are as follows:

1. Edit the workbook in Tableau Desktop (this is needed for a specific tool that is used)
1. Add the datasource to swap to as a new datasource in the workbook.
1. Navigate to a sheet that uses datasource to be replaced.
1. Right click on datasource to be replaced and select Replace Data Source...
1. In the dialog box ensure that datasource to be replaced is selected as Current and select new datasource for the Replacement and select ok
1. Check that the all of the fields swapped over to the new datasource are working and not showing an error- some may have a ! next to them and require replacing. Any manual field aliases may also need to be reapplied.
1. Right click on the datasource to be replaced and select close (to reduce un-needed clutter).
1. Publish the workbook.

Testing Tables from MR Databases in Tableau Prior to Merging

If you are working in Tableau using a report table from snowflake (created in DBT), you will likely need to update your table at some point. It is in your best interest to test these changes in Tableau prior to merging your MR, so that you can catch any problems with the updates before going through the process of formally requesting to merge the changes, and waiting for the data to be available.

There are some key lessons that the Data Team learned about testing our MR databases in Tableau, which we will share below.

Workflow

1. The author of the MR shares the MR database with you.

2. Open up a development copy of the workbook or data source you want to test the changes on.
 1. Make sure you are using a development copy and not working on the published data source! This is important because at some point in the future, when the MR is merged, you will no longer be able to access the data source that points to the MR db - even to update it to repoint to PROD. Hence it is important that you leave the original data source untouched to avoid this issue.
3. Open up the data connections pane.
4. Find the MR database in the left-hand connections window/dropdown.
5. Replace the PROD tables with the MR database tables.
6. Test your changes.
7. Close and exit without saving your changes, or un-do the changes to revert the dashboard back to it's original state and data source.
8. Merge the MR when you are satisfied with the changes - the MR database will disappear.

Forming a Connection

Once you have a local development copy of the data source, open up the data source connection pane where you would normally edit a data source.

On the left side is where you add new connections, and in the middle is where the tables that make up the workbook are visualized.

If you have been granted access to query the MR database which is attached to the merge request you are looking for, then you will be able to see it as an option in the dropdown for "Database".

Search this MR database for your desired tables. Create your data source as you normally would - either replacing existing tables with a test version, or bringing out new tables into the model and creating a join or relationship.

You can now test the tables that the MR would build right in your Tableau workbook, to make sure all of the changes will have the desired effect.

Saving Changes

You cannot save these changes that you are testing, because once the MR gets merged, the MR database you are using will disappear.

It is recommended you only test the logic and totals of the columns being added/changed, and not make any time-consuming dashboard changes that will not be able to be saved.

If you try saving the development copy that is pointing to your development data source, you will be unable to access that data source.

Avoiding Errors

To repeat: once your Merge Request gets merged, any Tableau Data Source which is trying to connect to that MR database will become inaccessible. You will not be able to even open the data source to edit it - in Cloud, Desktop, a duplicate version, or via any other method.

This is why it is recommended to work on a development copy of the data source only, and not in the published/production version of the data source at all.

Even if you are 'searching for' the MR database (pictured below) but not using this connection for any of the tables in the workbook, you will encounter errors.

Below is the error you will get if any leftover connections to the dropped database still exist in your data source. There is no workaround for this, you will need to replace the data source with an identical data source or, if you do not have an identical version, you will need to rebuild it and then 'Replace References' on most of the fields.

Final Testing Notes

Testing MR databases is a useful way to test changes before they get merged into production and save time. The use cases that work best for this are:

- Business logic changes that would affect a total number
- Quick changes to fields that would affect the view

It is not efficient to extensively test changes that would require many changes to the dashboard/ calculated fields, because you will not be able to save those changes for use with the updated table once the MR goes through.

Make sure to open a local, development copy of the workbook/data source prior to testing the MR database.

What are Relationships? (Tableau)

Relationships are a feature in Tableau that allow you to combine data from multiple tables for analysis without having to define join types. They offer a more flexible and performant way to work with multi-table data sources compared to traditional joins. Here are some key points about relationships in Tableau:

1. Dynamic and flexible: Relationships adapt to the specific fields and filters used in a visualization, optimizing queries for better performance.
2. Maintain data granularity: Unlike joins, relationships preserve the native level of detail in each related table, reducing data duplication and aggregation issues.
3. Multiple tables at different levels of detail: You can easily relate tables with different levels of granularity without worrying about fanout or incorrect aggregations.

4. Noodle diagrams: Relationships are represented visually as "noodles" connecting tables in the data model, making it easier to identify table associations.
5. Context-aware: Tableau only queries the tables and fields necessary for the current visualization, improving performance and reducing unnecessary data retrieval.
6. Easy to set up: Simply drag and drop tables onto the canvas and define relationships based on common fields between tables.
7. Compatibility with joins: You can still use traditional joins within a single logical table, allowing for a hybrid approach when needed.
8. Performance optimization: Relationships often result in better query performance compared to complex join scenarios, especially for large datasets.
9. Simplified data modeling: Relationships make it easier to create and maintain complex data models without requiring extensive knowledge of join types and their implications.
10. Improved data accuracy: By maintaining the native level of detail in each table, relationships help prevent accidental data loss or duplication that can occur with poorly designed joins.

When working with multi-table data sources in Tableau, consider using relationships as your default approach for combining tables, reserving joins for specific scenarios where more precise control over table combinations is required.

If you would like to see a simple example demonstrating how relationships work - with the SQL queries that the example produced, you can find an in-depth writeup [here](#).

Snowflake & Tableau Settings Rolename Embedding

In order to properly embed your credentials in connections (as described [here](#)), you will need to have two prerequisites:

1. You have set a 'default role' in Snowflake. You can follow the instructions [here](#) on how to set your default role if you have not done so already.
1. You have embedded your rolename into your Tableau settings. To do this, navigate to your Tableau homepage and look to the top right corner. Click on your profile dropdown (usually a circle with your initials), and click 'My Account Settings'. In the "Saved Credentials for Data Sources" section, scroll until you find Snowflake, and add your default role there.

SECTION: enterprise-data/platform/tableau/tableau-developer-guide/tableau-style-guide/_index.md

Overall Look and Feel

Tableau's robust visualization capability allows for more options and flexibility around customizing the look-and-feel for our dashboards. In order to create consistent front-end designs that are true to GitLab's branding, we recommend following the tips and tricks, along with resources, provided below:

GitLab Branding Resource

The in-depth branding guidelines at GitLab provides instructions on leveraging company-specific images and colors for upholding our brand image in our work. For information on how to use logo images, please refer to the following links:

- Core logo
- Logomark
- Branded lockups

The company branding resource site also provides excellent guidance around data visualization. For example, the color palettes are useful in creating accurate coloring to charts via hex codes:

- Color
- Charts

Please also refer to our Tableau Design folder (TBD) for files that can be used in adding branding elements to your Tableau dashboards. For example, our logo can be added to the upper left-hand corner of our dashboard, above the filter section that is using our GitLab color scheme.

Tableau Help

To create custom color palettes in Tableau, you can update the Preferences.tps file with the code (below under our Standard Color Palette section) to reflect your intended color scheme.

Please refer to the below sites for more instructions from Tableau around customizing texts and colors in workbooks:

- Creating Custom Palettes
- Format Text
- Use Custom Fonts

Choosing the Right Chart

Pie charts

Try to avoid using pie charts and use bar charts instead. The human eye is not well suited to easily grasp the difference in the sizes of the slices of the pie chart, a problem amplified with every added slice over two. If you must use pie charts, reserve them only for use cases where only two dimensional values need to be represented (although even there bar charts may be preferable).

For an in-depth explanation of the limitations of pie charts, please read Stephen Few's

excellent article.

Colors, Labels, and Data Values

Standard Color Palette

GitLab's brand approved colors can be found [here](#). Custom color palettes can be created in Tableau Desktop by with the following steps:

1. In the your laptop Documents folder go to My Tableau Repository and locate the Preferences.tps file.
2. Open the file with a preferred text editor such as TextEdit or Visual Studio Code.
3. Enter the desired colors for your palette into the Preferences file as follows:

```
<details>
```

```
<summary>Example palettes: Preferences.tps file</summary>
```

```
xml
```

```
<?xml version='1.0'?>
```

```
<workbook>
```

```
<preferences>
```

```
<color-palette name="GitLab Brand Design"
```